Slide 1: Title and Introduction This slide introduces the course material, which is based on the textbook "Computer Organization and Architecture," 10th Edition, authored by William Stallings.

Slide 2: Chapter 1 Overview The focus of Chapter 1 is on "Basic Concepts and Computer Evolution". This chapter lays the foundational terminology required to understand how computers are built and traces the historical timeline of computing technology to explain why modern systems are designed the way they are.

Slide 3: Computer Architecture Computer architecture refers to the attributes of a computing system that are explicitly visible to the programmer. These are the characteristics that have a direct and tangible impact on the logical execution of software programs.

- **Instruction Set:** The specific library of commands the processor understands, such as mathematical operations (add, subtract) or data movement commands.
- **Data Representation:** How the system formats and stores information, such as the number of bits used to represent data types like integers or floating-point numbers.
- **I/O Mechanisms:** The fundamental ways the system handles input and output operations.
- **Addressing Techniques:** The specific logical methods the processor uses to locate and access data within the computer's memory.

Slide 4: Computer Organization In contrast to architecture, computer organization refers to the operational units and their physical interconnections that actually realize the architectural specifications. These are the underlying hardware details that remain completely transparent (invisible) to the programmer.

- **Control Signals:** The electrical pulses that orchestrate and synchronize the timing of operations across the hardware.
- **Interfaces:** The physical connections and communication protocols between the computer and external peripherals, such as USB ports or display connectors.
- **Memory Technology:** The physical hardware used to store data, such as deciding whether a system uses DDR4 RAM, DDR5 RAM, or specific types of flash storage.

Slide 5: Comparing Architecture and Organization

This slide summarizes the distinction between the two concepts side-by-side.

- **Architecture** includes the logical attributes visible to the programmer, such as the instruction set, addressing techniques, and data type bit representations, which directly dictate how a program logically executes.
- **Organization** includes the operational units and hardware details that are transparent to the programmer, such as control signals, memory technology, and interfaces, which simply perform the heavy lifting to make the architecture function.

Slide 6: Architecture & Organization - Family Concept Many computer manufacturers utilize a "family" concept, offering various computer models that all share the exact same architecture but have distinct differences in their organization.

- For example, all processors in the Intel x86 family share the same basic architecture. This means a program written for an old Intel processor will still be understood by a brand-new one.
- The IBM System/370 family also shares a common architecture across its range.
- While the organizational hardware changes between versions (e.g., faster clock speeds, new manufacturing processes), retaining the same architecture ensures that a customer's older software continues to function perfectly on new machines.

Slide 7: IBM System 370 Architecture The IBM System/370 architecture is a prime example of the family concept, introduced in 1970 with a variety of models.

- It allowed customers to seamlessly upgrade to a more expensive, faster model without ever having to abandon their original software.
- As IBM introduced new models with vastly improved technology, they deliberately retained the original architecture to protect the massive software investments made by their customers.
- This specific architecture has been so successful that it has survived to the present day as the core architecture for IBM's modern mainframe product line.

Slide 8: Structure and Function A contemporary computer is an incredibly complex system that contains millions of elementary electronic components. To clearly describe and design such a system, one must recognize its hierarchical nature, treating the computer as a set of interrelated subsystems. This allows designers to deal with only a particular level of the system at a time. At each level, the designer is concerned with two things:

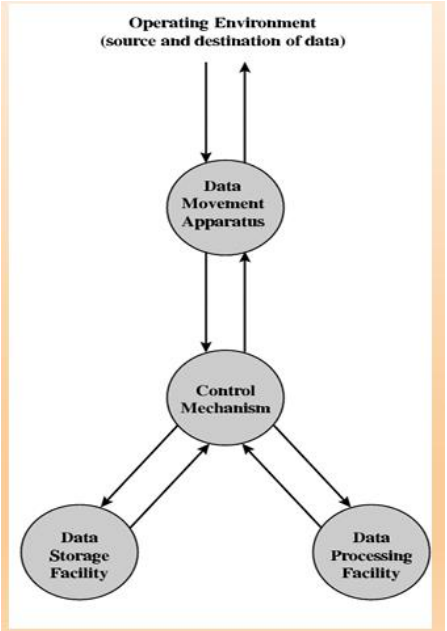
- **Structure:** The specific way in which the components relate to and connect with each other.
- **Function:** The operation of individual components as they work together as part of that larger structure.

Slide 9: The Four Basic Functions A computer is designed to perform four fundamental functions:

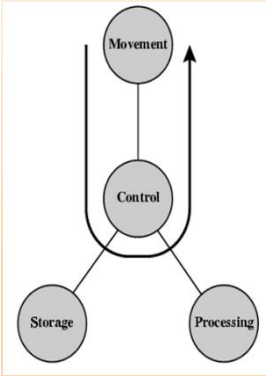
- **Data Processing:** Manipulating information, which can take a wide variety of forms and require a broad range of processing techniques (e.g., calculating math, rendering graphics).
- **Data Storage:** Retaining information, which includes short-term storage (like system RAM for active tasks) and long-term storage (like hard drives or SSDs for saved files).
- **Data Movement:** This covers Input/Output (I/O) when data is transferred to or from a directly connected peripheral device, as well as Data Communications when data is transmitted over longer distances to remote devices.
- **Control:** A centralized control unit manages the computer's internal resources and orchestrates the performance of all its functional parts in response to given instructions.

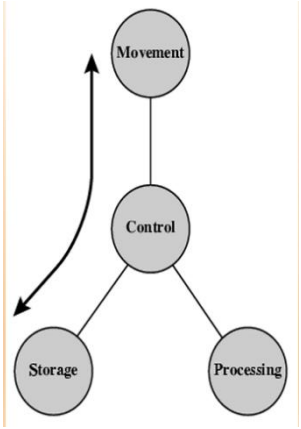
Slide 10: A Functional View of the Computer This slide provides a visual diagram showing how the computer interacts with the outside world, which is labeled as the "Operating Environment" (the source and destination of data).

- The system must be able to process data, store data, and move data between itself and this external environment.
- The diagram features a "Control Mechanism" at the center, which links to a "Data Movement Apparatus", a "Data Storage Facility", and a "Data Processing Facility".
- This central control unit acts as the brain, managing the computer's resources and the performance of these functional parts whenever instructions are received.

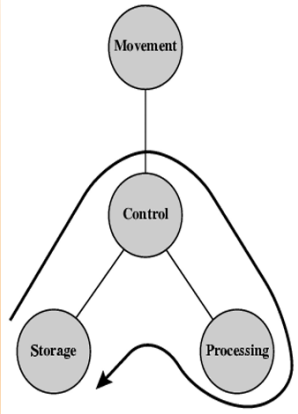


Slide 11: Operations (1) Data Movement This slide illustrates the first of the four possible types of operations: Data Movement. In this mode, the computer functions primarily as a data movement device. It simply transfers data from one peripheral or communication line to another. An example of this is typing on a keyboard, where the keystroke data is directly moved to and displayed on the screen. The diagram visually represents this by showing an arrow moving through the "Movement" and "Control" nodes, completely bypassing the "Storage" and "Processing" nodes.

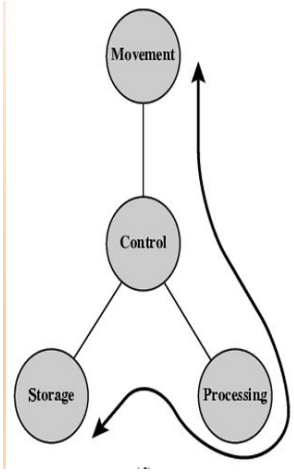




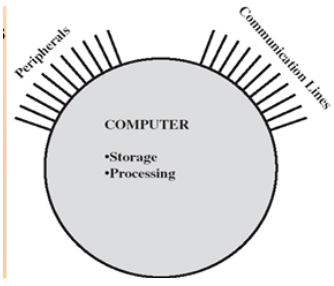
Slide 12: Operations (2) Storage The second operation type focuses on data storage. Here, the computer acts as a data storage device, transferring data from the external environment into the computer's storage for read or write purposes. A common everyday example is downloading a file from the Internet directly to your hard disk. The accompanying diagram shows the data flow originating from the external "Movement" node, passing through "Control", and terminating at the "Storage" node.



Slide 13: Operation (3) Processing from/to Storage This slide details operations that involve processing data that is already held within the system's storage. Instead of interacting with the external environment, the control unit fetches data from the storage unit, routes it to the processing facility, and then stores the resulting data back. An example provided is updating a bank statement, where existing numbers are retrieved, recalculated, and saved back to the disk. The diagram reflects this closed loop, showing a continuous internal path between "Storage", "Control", and "Processing".



Slide 14: Operation (4) Processing from Storage to I/O The final operational view combines processing with input/output (I/O) operations. This involves processing data as it is routed between the internal storage and the external environment. For instance, printing a bank statement requires taking data from storage, processing or formatting it, and moving it out to a peripheral printer. The diagram shows a path that starts at "Storage", goes through "Processing" and "Control", and finally exits via "Movement".

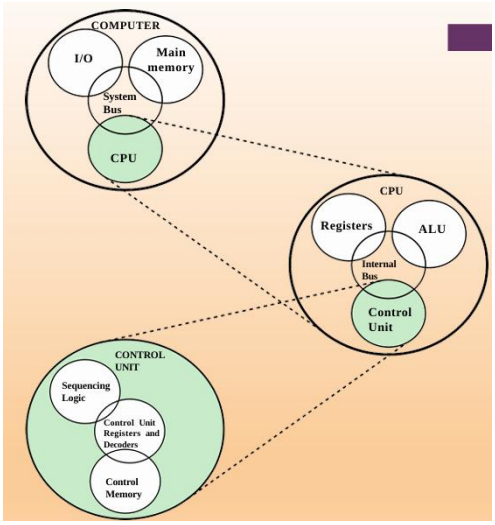


Slide 15: Computer Structure - Top Level At the top level, a computer consists of four main structural components:

- **Central Processing Unit (CPU):** Often referred to simply as the processor, this component controls the operation of the computer and performs all its data processing functions.
- **Main Memory:** This component is responsible for storing data.
- **I/O:** This component handles moving data between the computer and its external environment, which includes peripherals and communication lines.
- **System Interconnection:** This represents the underlying mechanisms that provide for communication among the CPU, main memory, and I/O components.

Slide 16: Structure - A Top-Down View This slide provides a visual hierarchy of the computer's structure (Figure 1.1).

- It shows the top-level System Bus connecting the CPU, Main Memory, and I/O.
- It then zooms into the CPU, revealing its internal components: Registers, the ALU, and the Control Unit, all linked by an internal CPU bus.
- It further zooms into the Control Unit, showing its sub-components like Sequencing Logic, Control Unit Registers and Decoders, and Control Memory.
- This top-down view emphasizes the hierarchical complexity of the system, demonstrating how larger components are built from smaller, interconnected subsystems.



Slide 17: Main Structural Components Overview This slide visually reiterates the four main structural components mentioned previously.

- **CPU:** The core that controls operations and performs data processing.
- **Main Memory:** The component that stores data.
- **I/O:** The interface that moves data between the computer and its external environment.
- **System Interconnection:** The mechanism that provides communication among the CPU, main memory, and I/O.

Slide 18: Major Structural Components of the CPU Diving deeper into the CPU itself, this slide outlines its major internal structures:

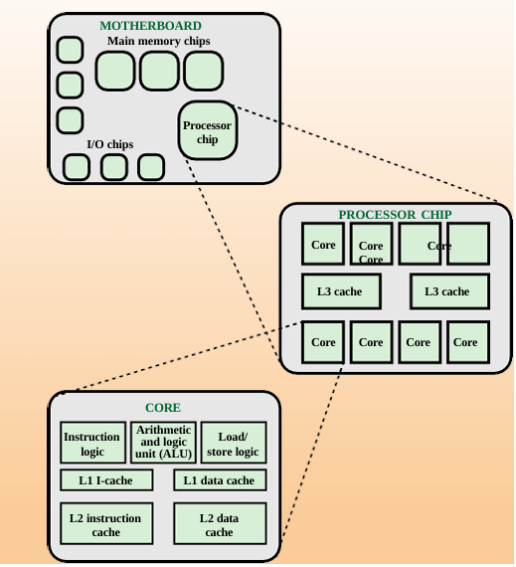
- **Control Unit:** The director of the CPU; it controls the operation of the CPU and, consequently, the entire computer.
- **Arithmetic and Logic Unit (ALU):** The workhorse component that actually performs the computer's data processing, including mathematical and logical operations.
- **Registers:** Small, exceptionally fast memory locations that provide internal storage directly within the CPU for immediate data access during execution.
- **CPU Interconnection:** A mechanism that facilitates vital communication among the control unit, ALU, and registers.

Slide 19: Multicore Computer Structure This slide shifts the focus to modern processor terminology and multicore designs.

- **Central Processing Unit (CPU):** The portion of the computer that fetches and executes instructions. It consists of an ALU, a control unit, and registers. In older systems with a single processing unit, this was referred to simply as a processor.
- **Core:** An individual processing unit located on a processor chip. A single core may be entirely equivalent in functionality to an older CPU on a single-CPU system. Specialized processing units are also referred to as cores.
- **Processor:** A physical piece of silicon containing one or more cores. It is the component that interprets and executes instructions. It is specifically referred to as a multicore processor if it contains multiple cores.

Slide 20: Simplified View of a Multicore Computer This slide presents Figure 1.2, illustrating the physical hierarchy of a multicore system.

- At the highest level is the **Motherboard**, which houses the Processor chip, Main memory chips, and I/O chips.
- Zooming in on the **Processor Chip**, the diagram reveals multiple Cores and shared L3 cache memory.
- Zooming further into an individual **Core**, it details the Instruction logic, ALU, Load/store logic, and dedicated L1 (instruction and data) and L2 caches.



Slide 21: Cache Memory

Cache memory consists of multiple layers of high-speed memory situated between the processor and the main memory.

- It is physically smaller but operates significantly faster than standard main memory (RAM).
- Its primary function is to speed up memory access by temporarily storing data from main memory that the processor is likely to need in the near future. For example, if a program loops the same calculation, the data for that calculation stays in the cache for instant access.
- Systems achieve greater performance improvements by utilizing multiple levels of cache.
- Level 1 (L1) cache is the closest to the processing core and the fastest, while additional levels like Level 2 (L2) and Level 3 (L3) are progressively farther from the core, slightly slower, but larger in capacity.

Slide 22: Motherboard with Two Intel Quad-Core Xeon Processors

This slide displays a detailed hardware diagram (Figure 1.3) of a highly capable server-grade motherboard.

- It features sockets for two Quad-Core Intel Xeon Processors, each equipped with integrated memory controllers.
- The board includes six-channel DDR3-1333 memory interfaces capable of supporting up to 48GB of RAM.
- Various peripheral connections are highlighted, including Serial ATA/300 (SATA) interfaces for hard drives, internal and external USB 2.0 ports, a VGA video output, and two Gigabit Ethernet ports (10/100/1000Base-T).
- Internal expansion and management components are also shown, such as multiple PCI Express connectors, the system clock, the BIOS chip, and power connectors.

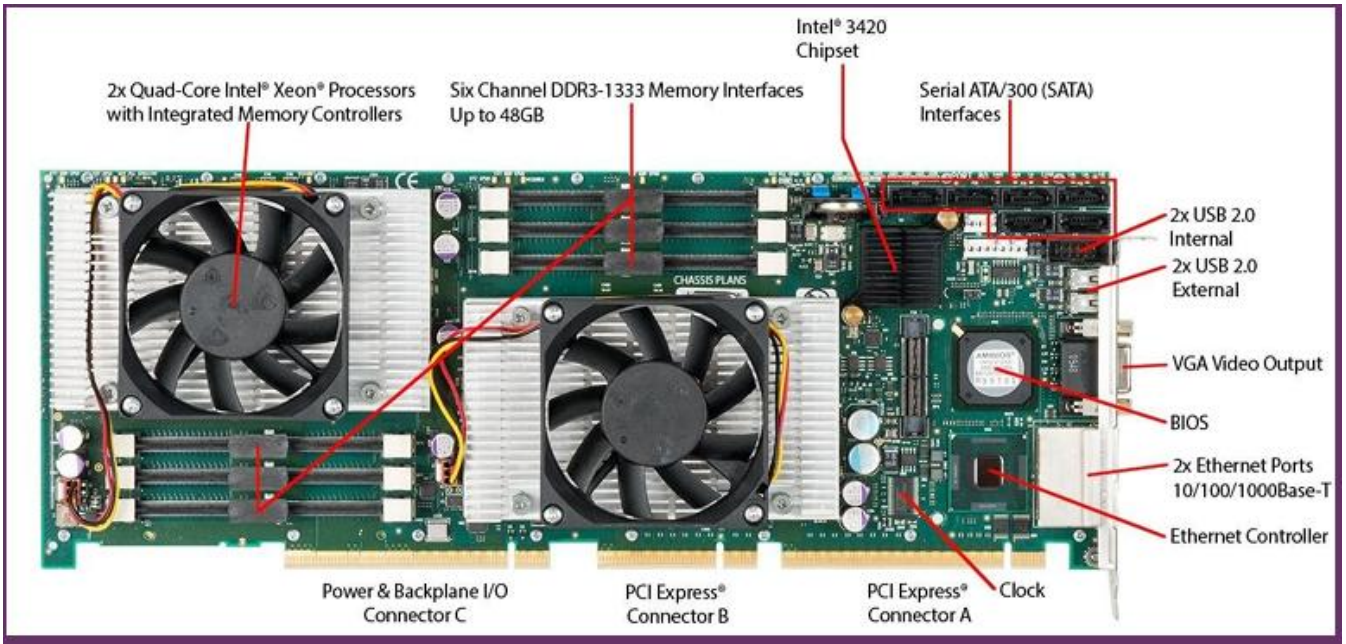


Figure 1.3
Motherboard with Two Intel Quad-Core Xeon Processors

Slide 23: Computer Generations

This table (Table 1.2) categorizes the evolution of computers into six distinct generations based on their underlying technology and processing speeds.

- **Generation 1 (1946–1957):** Built using vacuum tubes, these machines could perform approximately 40,000 operations per second.
- **Generation 2 (1957–1964):** Transitioned to transistors, boosting speeds to around 200,000 operations per second.
- **Generation 3 (1965–1971):** Utilized small and medium scale integration, reaching 1,000,000 operations per second.
- **Generation 4 (1972–1977):** Featured large scale integration (LSI), achieving 10,000,000 operations per second.
- **Generation 5 (1978–1991):** Used very large scale integration (VLSI), reaching 100,000,000 operations per second.
- **Generation 6 (1991–present):** Relies on ultra large scale integration (ULSI), performing over 1,000,000,000 operations per second.

Generation	Approximate Dates	Technology	Typical Speed (operations per second)
1	1946–1957	Vacuum tube	40,000
2	1957–1964	Transistor	200,000
3	1965–1971	Small and medium scale integration	1,000,000
4	1972–1977	Large scale integration	10,000,000
5	1978–1991	Very large scale integration	100,000,000
6	1991-	Ultra large scale integration	>1,000,000,000

Slide 24: History of Computers - First Generation: Vacuum Tubes

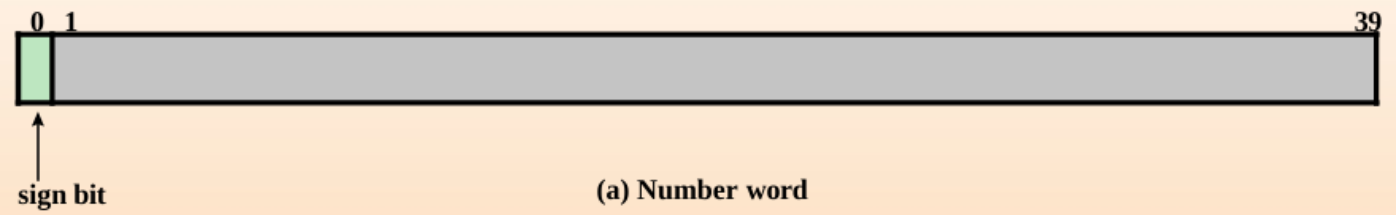
The first generation of computers relied on vacuum tubes for both digital logic elements and memory storage. These tubes were large, fragile glass bulbs that controlled electrical current in a high vacuum.

- A key example from this era is the IAS computer.
- Its fundamental design was based on the stored program concept, an architectural breakthrough attributed to mathematician John von Neumann. This concept meant that both instructions and data were stored in the same memory space.
- The idea was first published in 1945 for the EDVAC project.
- Design of the IAS computer began at the Princeton Institute for Advanced Studies (IAS) and was ultimately completed in 1952.
- This machine served as the prototype for all subsequent general-purpose computers.

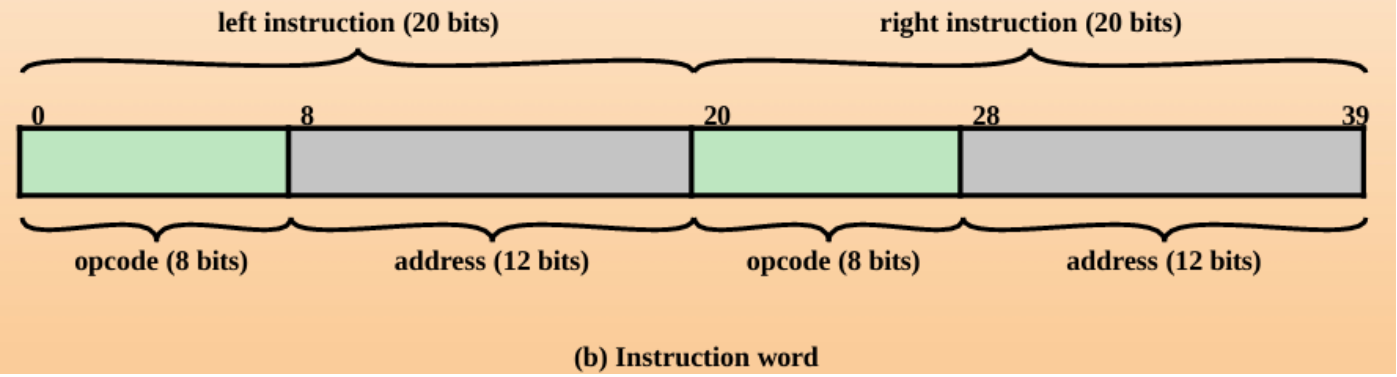
Slide 25: IAS Memory Formats

This slide details how the IAS computer formatted its data and instructions within its 40-bit memory structure (Figure 1.7).

- **(a) Number word:** When storing a mathematical value, the first bit (bit 0) was designated as the sign bit (indicating positive or negative), while the remaining 39 bits represented the numerical value.



- **(b) Instruction word:** A single 40-bit word could hold two separate 20-bit instructions (a left instruction and a right instruction).
- Within each 20-bit instruction, the first 8 bits were the "opcode" (the operational command, like "add" or "load"), and the remaining 12 bits represented the memory address associated with that operation.



Slide 26: IAS Registers

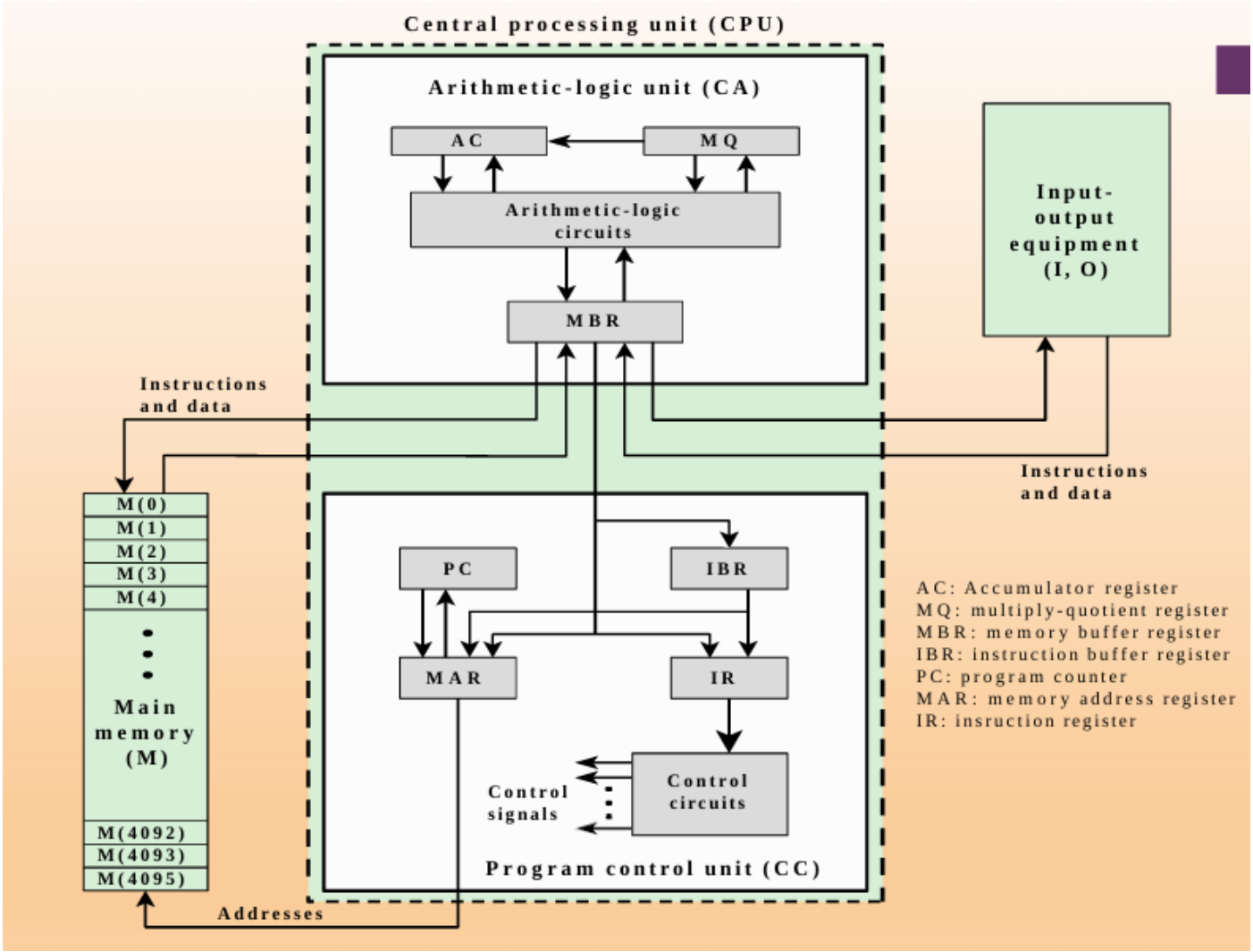
The IAS computer utilized specific hardware registers within the CPU to temporarily hold data and instructions during execution.

- **Memory Buffer Register (MBR):** Holds a word that is either waiting to be stored in memory, sent to the I/O unit, or has just been received from memory or the I/O unit.
- **Memory Address Register (MAR):** Specifies the exact memory address where a word is to be written from the MBR or read into the MBR.
- **Instruction Register (IR):** Contains the specific 8-bit opcode of the instruction currently being executed.
- **Instruction Buffer Register (IBR):** Temporarily holds the right-hand instruction from a retrieved memory word so it is ready when the left-hand instruction finishes.
- **Program Counter (PC):** Keeps track of the execution sequence by containing the memory address of the next instruction pair to be fetched.
- **Accumulator (AC) and Multiplier Quotient (MQ):** These two registers are employed by the ALU to temporarily hold mathematical operands and the final results of operations.

Slide 27: IAS Structure

This schematic (Figure 1.6) visually connects the components discussed in the previous slides.

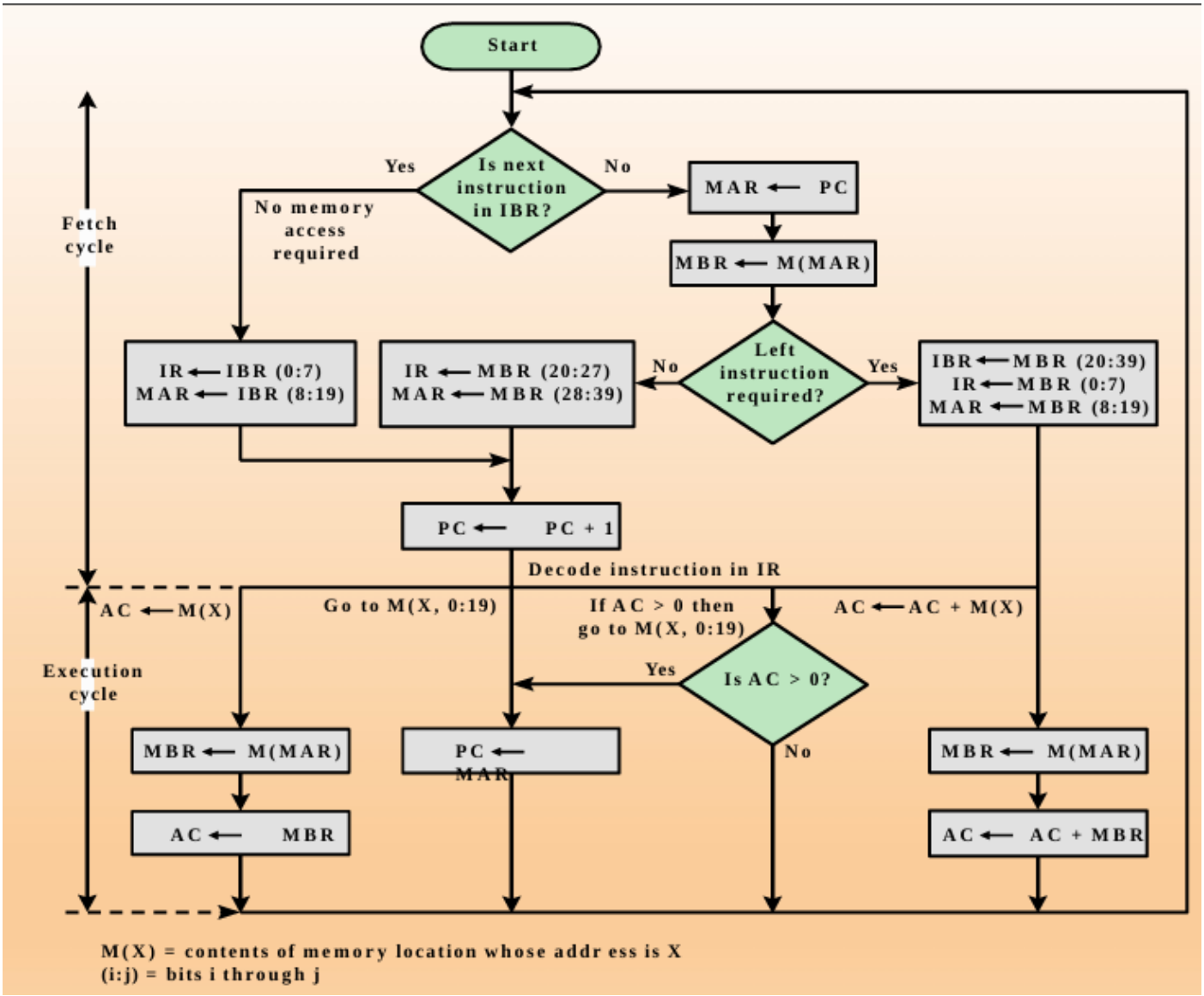
- It illustrates the Main memory (M) containing addresses ranging from M(0) to M(4095).
- The CPU is divided into the Arithmetic-logic unit (containing the AC, MQ, arithmetic circuits, and MBR) and the Program control unit (containing the PC, MAR, IR, IBR, and control circuits).
- The diagram shows how data and instructions flow between the memory, the CPU's internal registers, and the Input-output equipment.



Slide 28: Partial Flowchart of IAS Operation

This flowchart (Figure 1.8) breaks down the fundamental machine cycle of the IAS computer into two main phases: the Fetch cycle and the Execution cycle.

- **Fetch Cycle:** The system checks if the next instruction is already waiting in the Instruction Buffer Register (IBR). If not, it fetches a new instruction pair from memory, places it in the MBR, routes the left instruction to the Instruction Register (IR), and queues the right instruction in the IBR. The Program Counter (PC) is then incremented.
- **Execution Cycle:** Once an instruction is loaded into the IR, the control circuitry decodes the opcode and performs the operation. The flowchart shows branching paths for different types of commands, such as data movement or conditional logic based on the Accumulator's value.



Slide 29: The IAS Instruction Set

The IAS computer featured a remarkably concise instruction set totaling only 21 instructions, which are categorized in Table 1.1.

- **Data Transfer:** Instructions for moving data between memory, the Accumulator, and the MQ register (e.g., LOAD MQ, STOR M(X)).

- **Unconditional Branch:** Commands to jump directly to a new instruction address regardless of current conditions (e.g., JUMP M(X,0:19)).
- **Conditional Branch:** Commands to jump to a new address only if a specific condition is met, such as the value in the accumulator being a non-negative number.
- **Arithmetic:** Basic mathematical operations like ADD, SUB, MUL (multiply), DIV (divide), as well as left and right bit-shifts (LSH, RSH).
- **Address Modify:** Specialized commands that allowed the system to replace the address fields of instructions stored in memory.

Instruction Type	Opcode	Symbolic Representation	Description
	00001010	LOAD MQ	Transfer contents of register MQ to the accumulator AC
Data transfer	00001001	LOAD MQ,M(X)	Transfer contents of memory location X to MQ
	00100001	STOR M(X)	Transfer contents of accumulator to memory location X
	00000001	LOAD M(X)	Transfer M(X) to the accumulator
	00000010	LOAD –M(X)	Transfer –M(X) to the accumulator
	00000011	LOAD |M(X)|	Transfer absolute value of M(X) to the accumulator
	00000100	LOAD –|M(X)|	Transfer –|M(X)| to the accumulator
Unconditional branch	00001101	JUMP M(X,0:19)	Take next instruction from left half of M(X)
	00001110	JUMP M(X,20:39)	Take next instruction from right half of M(X)
	00001111	JUMP+ M(X,0:19)	If number in the accumulator is nonnegative, take next instruction from left half of M(X)
Conditional branch		*JUMP+ M(X,20:39)*	*If number in the accumulator is nonnegative, take next instruction from right half of M(X)*
Arithmetic	00000101	ADD M(X)	Add M(X) to AC; put the result in AC
	00000111	ADD |M(X)|	Add |M(X)| to AC; put the result in AC
	00000110	SUB M(X)	Subtract M(X) from AC; put the result in AC
	00001000	SUB |M(X)|	Subtract |M(X)| from AC; put the remainder in AC
	00001011	MUL M(X)	Multiply M(X) by MQ; put most significant bits of result in AC, put least significant bits in MQ
	00001100	DIV M(X)	Divide AC by M(X); put the quotient in MQ and the remainder in AC
	00010100	LSH	Multiply accumulator by 2; i.e., shift left one bit position
	00010101	RSH	Divide accumulator by 2; i.e., shift right one position
Address modify	00010010	STOR M(X,8:19)	Replace left address field at M(X) by 12 rightmost bits of AC
	00010011	STOR M(X,28:39)	Replace right address field at M(X) by 12 (Tab rightmost bits of AC

Slide 30: History of Computers - Second Generation: Transistors

The transition to the second generation of computing was marked by the replacement of vacuum tubes with transistors.

- Transistors are solid-state electronic devices manufactured from semiconductor materials like silicon.
- They offered massive advantages: they were physically much smaller, vastly cheaper to produce, and dissipated significantly less heat than vacuum tubes, preventing the massive hardware burnouts common in early computers.
- The transistor was invented at Bell Labs in 1947.
- Despite being invented in the 1940s, it took until the late 1950s for fully transistorized computers to become commercially available to the public.

Slide 31: The First Transistor

This slide presents historical photographs of foundational computing hardware.

- **Image (a)** displays the very first transistor, developed by Texas Instruments.
- **Image (b)** showcases the first integrated circuit, a property of AT&T Archives.
- These physical inventions, stemming from work at Bell Labs around 1946, represent the critical shift from bulky, heat-intensive vacuum tubes to small, solid-state electronics, making modern computing physically possible.

Slide 32: Second Generation Computers

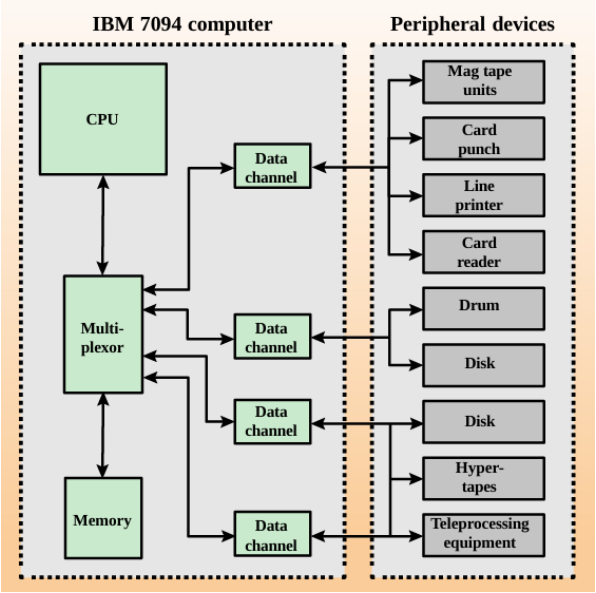
The shift to transistors defined the second generation of computers and brought about major structural and software advancements.

- Hardware improved by introducing far more complex Arithmetic and Logic Units (ALUs) and Control Units, allowing the computer to process more advanced calculations efficiently.
- This era saw the introduction of high-level programming languages, moving developers away from tedious machine code into more human-readable programming.
- System software was provided to manage basic operations, granting the ability to load programs automatically, route data to external peripherals smoothly, and utilize shared libraries for common mathematical computations.

Slide 33: IBM 7094 Configuration

This slide provides a block diagram (Figure 1.9) illustrating the architecture of the IBM 7094, a prominent second-generation computer.

- The system heavily utilized a **Multiplexor** to manage data flow between the CPU, the Main Memory, and various external devices.
- Communication with peripheral devices was handled through dedicated **Data channels**.
- The wide array of supported peripheral devices reflects the growing utility of computers, including magnetic tape units, card punches, line printers, card readers, drums, disks, hyper-tapes, and teleprocessing equipment.



Slide 34: History of Computers - Third Generation (Integrated Circuits)

The third generation was defined by the transition from discrete components to integrated circuits.

- The integrated circuit was invented in 1958.
- Prior to this, discrete components (individual, self-contained transistors) were manufactured separately, packaged in their own containers, and then painstakingly soldered or wired together onto masonite-like circuit boards.
- This older manufacturing process was incredibly expensive, prone to physical failure, and cumbersome to scale.
- The two most significant computers that emerged during this third-generation era were the IBM System/360 and the DEC PDP-8.

Slide 35: Integrated Circuit (IC) Inventions

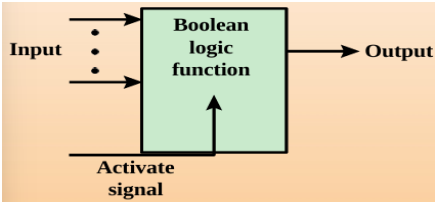
This slide highlights the specific milestones of the integrated circuit.

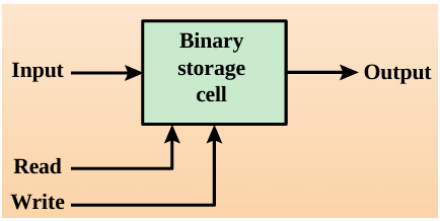
- It credits Jack Kilby at Texas Instruments for the monumental invention of the IC in 1958.
- It also features an image of early bipolar logic circuitry from the 1960s, specifically an ECL 3-input Gate produced by Motorola in 1966. By combining multiple transistors and logic gates onto a single physical piece of material, manufacturers drastically reduced the size and cost of computers.

Slide 36: Fundamental Computer Elements

This slide breaks down the most basic building blocks of computer operations into two fundamental elements: gates and memory cells (Figure 1.10).

- **(a) Gate:** This represents the processing element. It takes one or more inputs, evaluates a Boolean logic function (like AND, OR, NOT) when triggered by an activate signal, and produces a resulting output.
- **(b) Memory Cell:** This represents the storage element. It is a binary storage cell that takes an input, utilizes specific read and write control signals to determine its action, and outputs the stored binary data when requested.



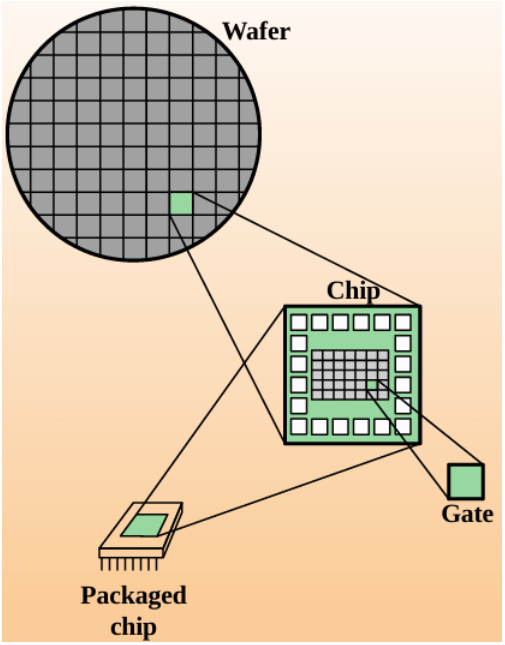


Slide 37: How Integrated Circuits Function

An integrated circuit combines the fundamental elements from the previous slide onto a single piece of semiconductor material.

- **Data storage** is provided directly by the embedded memory cells.
- **Data processing** is executed by the embedded logic gates.
- **Data movement** occurs via microscopic conductive paths that connect memory to memory, and memory through gates.
- **Control signals** are carried along these same interconnections to dictate operations.
- These components (transistors, resistors, and conductors) are fabricated directly out of a semiconductor, typically silicon. Instead of building components one by one, millions of transistors can be produced simultaneously on a single wafer of silicon, which are then connected by metallization to form a complete circuit.

Slide 38: Relationship Among Wafer, Chip, and Gate



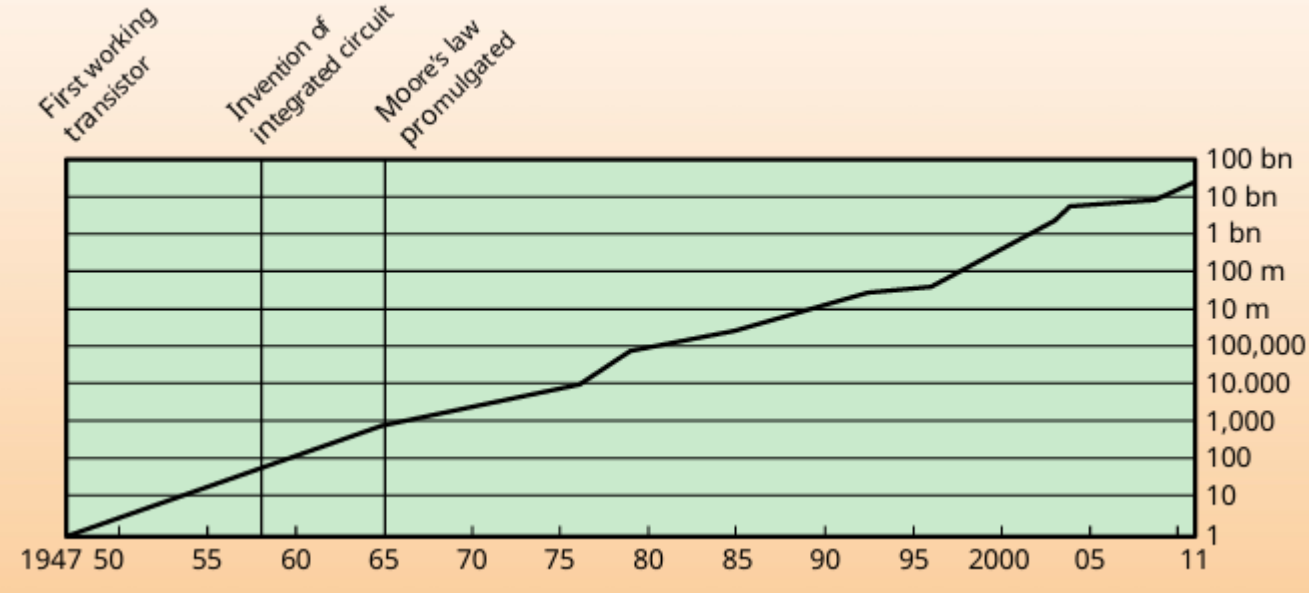
This slide visually scales down the manufacturing process of integrated circuits (Figure 1.11).

- Production starts with a large, circular silicon **Wafer**.
- This wafer is diced into many smaller square pieces, each known as a **Chip** (or die).
- The chip is then placed inside a protective casing with metal pins, becoming a **Packaged chip** ready to be installed on a motherboard.
- Zooming deep into the chip reveals a massive grid, where each microscopic square represents an individual logic **Gate**.

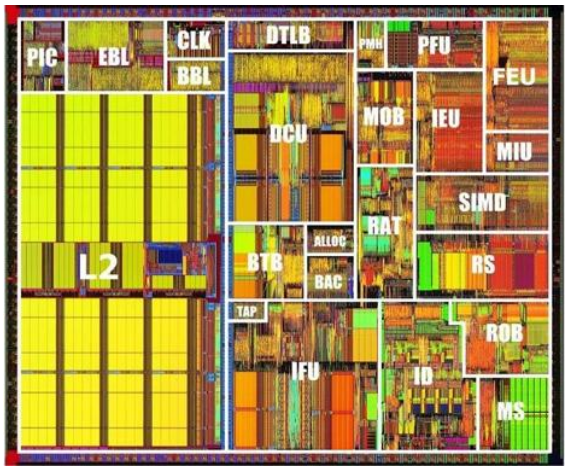
Slide 39: Moore's Law Growth Graph

This graph (Figure 1.12) visually represents Moore's Law by tracking the explosive growth in transistor count on integrated circuits (specifically DRAM memory) over time.

- The timeline begins with the first working transistor in 1947 (containing just 1 transistor) and marks the invention of the integrated circuit in 1958.
- Moore's Law was formally promulgated in 1965, predicting exponential growth.
- The logarithmic y-axis shows how chip capacity scaled dramatically from thousands of transistors in the 1970s, to millions in the 1990s, surpassing 1 billion transistors by the mid-2000s, and approaching 100 billion by 2011.



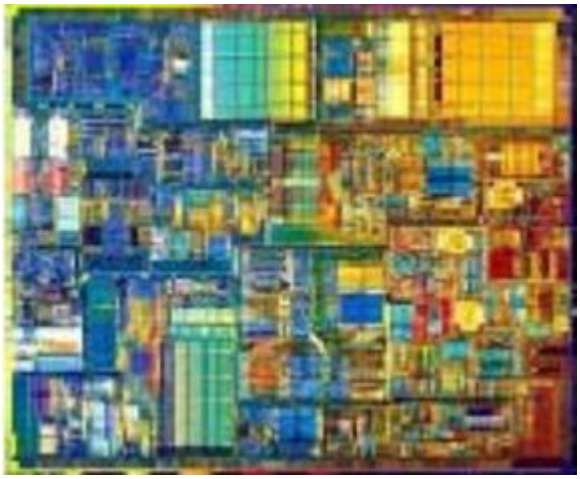
Slide 40: Pentium III Architecture



This slide presents a microscopic die shot of an Intel Pentium III processor, mapping out its internal physical layout.

- It highlights the sheer density of third-generation microprocessors, packing **28.1 million transistors** onto a die area of just 106 square millimeters.
- It was manufactured using a 0.18-micron process with 6-layer metal CMOS.
- Architecturally, it features a 3-way superscalar design, includes 256K of L2 cache built directly onto the chip, and utilizes a 133 MHz I/O bus for system communication. The color-coded image maps where specific functions—like the Data Cache Unit (DCU), Instruction Fetch Unit (IFU), and L2 cache—physically reside on the silicon.

Slide 41: Pentium IV



This slide gives a microscopic die shot and technical breakdown of the Intel Pentium IV, showing the rapid advancement from the Pentium III.

- **0.18-Micron Process Technology:** The initial versions of the Pentium 4 were built using a 0.18-micron manufacturing process. This means the transistors were incredibly small, allowing Intel to pack 42 million transistors onto the chip.
    - These early models ranged in clock speed from 1.4 GHz up to 2 GHz.
    - The 1.4 GHz and 1.5 GHz models were introduced on November 20, 2000, while the 1.9 GHz and 2 GHz models dropped on August 27, 2001.
    - They featured a 400 MHz system bus speed, SSE2 SIMD extensions (great for multimedia), and 256 KB of L2 cache integrated directly into the chip. They were primarily used in desktops and entry-level workstations.

- **0.13-Micron Process Technology:** Intel later shrank the manufacturing process to 0.13 microns, allowing them to squeeze 55 million transistors onto the die.
    - Introduced on January 7, 2002, these ran faster (2 GHz to 2.53 GHz) and doubled the L2 cache to 512 KB.

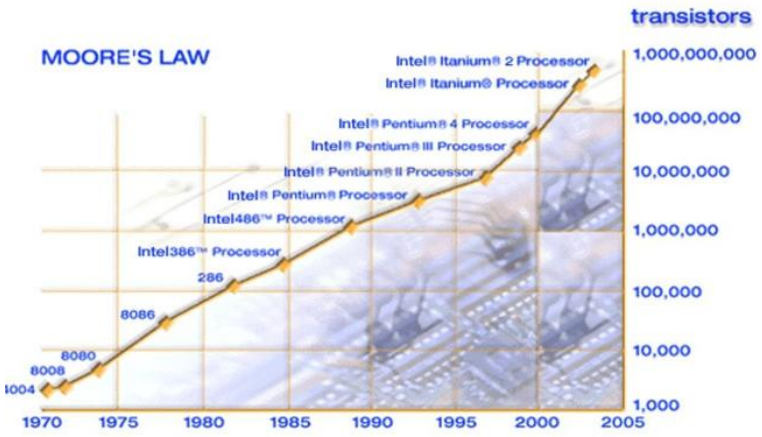
Slide 42: Moore's Law

This is one of the most famous principles in computer science, named after Gordon Moore, a co-founder of Intel, who made a groundbreaking observation in 1965.

- **The Observation:** Moore noticed that the number of transistors that could be squeezed onto a single integrated circuit was doubling every single year.
- **The Adjustment:** In the 1970s, this blistering pace slowed slightly to a doubling every 18 months, but remarkably, the industry has sustained that rate ever since.
- **Consequences of Moore's Law:** This exponential growth caused massive ripple effects across the world:
    - The cost of computer logic and memory circuitry plummeted dramatically.
    - Because components got smaller, the electrical path length shortened, which increased the operating speed of the processors.
    - Computers became physically smaller, making them much more convenient to use in everyday environments (leading to laptops and smartphones).
    - Smaller chips meant a massive reduction in power consumption and cooling requirements.
    - It also led to fewer interchip connections, meaning hardware became more reliable.

Slide 43: Change in Microprocessors (Graph)

This slide visualizes Moore's Law with a line graph plotting Intel's microprocessor history from 1970 to 2005.



- The Y-axis is on a logarithmic scale, starting at 1,000 transistors and jumping all the way up to 1,000,000,000 (1 billion).
- You can clearly see the steady, exponential climb starting with the Intel 4004 and 8008 in the early 1970s.
- The line passes through legendary chips like the 8080, 8086, 286, 386, and 486 throughout the 70s and 80s.
- In the 1990s and 2000s, the graph hits the multi-million transistor mark with the Pentium, Pentium II, Pentium III, Pentium 4, and ultimately reaches toward the billion mark with the Itanium and Itanium 2 processors.

Slide 44: IBM System/360

This slide introduces one of the most important mainframe computers in history, announced in 1964.

- **A Bold Move:** The System/360 product line was intentionally designed to be completely incompatible with older IBM machines. This was a massive risk, but it paid off, becoming the

success of the decade and cementing IBM as the overwhelmingly dominant computer vendor in the world.
- **Lasting Legacy:** The underlying architecture of the System/360 was so well thought out that it remains the core architecture of IBM's mainframe computers to this very day.
- **The "Family" Concept:** It was the industry's first planned "family" of computers. All models within the 360 series were compatible, meaning a program written for a cheaper, slower model could be executed flawlessly on a faster, more expensive model in the series.

Slide 45: Family Characteristics

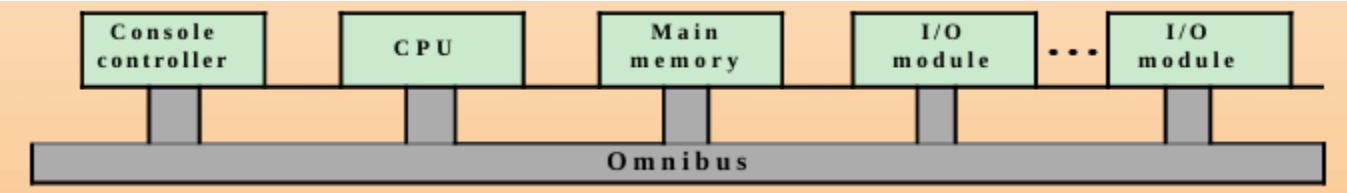
This slide dives deeper into why the IBM System/360's "family" concept was both novel and extremely successful.

- It allowed a customer with modest needs and a tight budget to start by purchasing a relatively inexpensive model. As their business grew, they could upgrade their hardware without throwing away all their expensive software.
- **The Upward Progression:** The slide shows a tiered list of characteristics that remained constant or increased as a customer moved up the family line:
  - They all shared a similar or identical instruction set and operating system.
  - As you upgraded, you gained an increasing number of I/O ports.
  - You gained increasing memory size.
  - You gained increasing processing speed.
  - Naturally, this all came with an increasing financial cost.

Slide 46: The Minicomputer Era (PDP-8)

While IBM was busy dominating the massive mainframe market, DEC (Digital Equipment Corporation) created a totally different revolution in the same year.

- **The PDP-8:** Shipped the same year as the System/360, this machine was dubbed a "minicomputer" by the industry (jokingly named after the miniskirt, which was popular at the time).
- **Size and Cost Revolution:** At a time when average computers required dedicated, air-conditioned rooms, the PDP-8 was small enough to sit on top of a lab bench or be built directly into other equipment. While it couldn't match a mainframe's power, it only cost $16,000. This was incredibly cheap compared to the hundreds of thousands of dollars an IBM 360 cost, making it affordable enough for individual lab technicians to have their own machine.
- **Omnibus Structure:** The diagram (Figure 1.13) shows its innovative "Omnibus" structure, where the CPU, Main Memory, Console Controller, and various I/O modules all connected to a single, centralized communication bus.



Slide 47: Later Generations

This slide visually introduces the terminology used to describe the ever-increasing density of integrated circuits that followed the third generation.

- **LSI (Large Scale Integration):** Representing chips that fit thousands of logic gates.

- **VLSI (Very Large Scale Integration):** The next leap, fitting tens to hundreds of thousands of gates onto a single chip.
- **ULSI (Ultra Large Scale Integration):** The modern standard, packing millions to billions of gates onto a single piece of silicon.
- The slide also points out that these advancements directly paved the way for modern **Semiconductor Memory** and the birth of **Microprocessors**.

Slide 48: Semiconductor Memory

Before the 1970s, computers used "magnetic core memory," which was bulky, expensive, and required literal wires woven through magnetic rings. This slide covers the semiconductor revolution that replaced it.

- **The Breakthrough:** In 1970, Fairchild produced the first relatively capacious semiconductor memory. The chip was about the size of a single old magnetic core, but it could hold 256 bits of memory, was non-destructive (reading the data didn't erase it), and was much faster than core memory.
- **The Tipping Point:** By 1974, the price per bit of this new semiconductor memory dropped below the price of the old core memory, effectively killing the old technology.
- **Rapid Evolution:** Since 1970, semiconductor memory has gone through 13 generations of development. Each successive generation has provided roughly four times the storage density of the previous one, alongside drastically declining costs per bit and faster access times. These developments radically changed the nature of computers in less than a decade.

Slide 49: Microprocessors

This slide covers the origin of the "CPU on a chip"—the microprocessor.

- As transistor density continued to rise, manufacturers could place more elements on a single chip, meaning fewer chips were needed to build a computer's processor. Eventually, they hit the holy grail: putting the *entire* CPU on one chip.
- **1971 - Intel 4004:** Intel developed the 4004, the very first chip to contain all the components of a CPU on a single piece of silicon. This marked the official birth of the microprocessor.
- **1972 - Intel 8008:** Just a year later, Intel developed the 8008, which was the world's first 8-bit microprocessor (the 4004 was only 4-bit).
- **1974 - Intel 8080:** Intel followed up with the 8080, recognized as the first truly general-purpose microprocessor. It was significantly faster, featured a much richer instruction set, and had a vastly larger addressing capability than its predecessors.

Slide 50: Evolution of Intel Microprocessors (1970s)

This slide presents a data table (Table (a) 1970s Processors) detailing the raw specs of Intel's earliest processor lineup, showing exactly how fast the tech was moving in just a single decade.

- **4004 (1971):** Clocked at 108 kHz, with a 4-bit bus width, featuring 2,300 transistors on a 10-micron process, and capable of addressing a mere 640 Bytes of memory.
- **8008 (1972):** Stayed at 108 kHz but bumped up to an 8-bit bus. Transistor count jumped to 3,500 on an 8-micron process, and memory capacity leaped to 16 KB.
- **8080 (1974):** A massive speed bump to 2 MHz (still an 8-bit bus). Transistors nearly doubled to 6,000 on a 6-micron process, addressing 64 KB of memory.
- **8086 (1978):** A huge leap to a 16-bit bus, with clock speeds ranging from 5 to 10 MHz. It packed 29,000 transistors on a 3-micron process and could address a full 1 Megabyte (MB) of memory.

- **8088 (1979):** A budget variant of the 8086. It shared the 5 and 8 MHz speeds and the 1 MB memory capacity, but utilized an 8-bit external bus to maintain compatibility with older, cheaper motherboard hardware.

	4004	8008	8080	8086	8088
Introduced	1971	1972	1974	1978	1979
Clock speeds	108 kHz	108 kHz	2 MHz	5 MHz, 8 MHz, 10 MHz	5 MHz, 8 MHz
Bus width	4 bits	8 bits	8 bits	16 bits	8 bits
Number of transistors	2,300	3,500	6,000	29,000	29,000
Feature size (µm)	10	8	6	3	6
Addressable memory	640 Bytes	16 KB	64 KB	1 MB	1 MB

	80286	386TM DX	386TM SX	486TM DX CPU
Introduced	1982	1985	1988	1989
Clock speeds	6 MHz - 12.5 MHz	16 MHz - 33 MHz	16 MHz - 33 MHz	25 MHz - 50 MHz
Bus width	16 bits	32 bits	16 bits	32 bits
Number of transistors	134,000	275,000	275,000	1.2 million
Feature size (µm)	1.5	1	1	0.8 - 1
Addressable memory	16 MB	4 GB	16 MB	4 GB
Virtual memory	1 GB	64 TB	64 TB	64 TB
Cache	—	—	—	8 kB

Slide 51: Evolution of Intel Microprocessors (1980s)

This slide tracks the rapid technological progression of Intel's processors throughout the 1980s, highlighting the shift into the 32-bit era.

- **The Processors:** This decade introduced the 80286 (1982), 386 DX (1985), 386 SX (1988), and the highly advanced 486 DX (1989).
- **Speed and Width:** Clock speeds surged from a modest 6–12.5 MHz on the 80286, up to 25–50 MHz on the 486 DX. The physical data bus width also doubled from 16 bits to 32 bits, allowing the CPU to process twice as much data per cycle.
- **Transistor Density:** The manufacturing feature size shrank from 1.5 microns down to 0.8–1 micron, allowing transistor counts to jump from 134,000 to 1.2 million (the 486 was the first x86 chip to break the one-million transistor mark).
- **Memory Capabilities:** Addressable physical memory expanded from 16 MB to 4 GB, while virtual memory management skyrocketed from 1 GB up to an incredible 64 Terabytes (TB). Furthermore, the 486 DX introduced 8 kB of built-in Cache memory directly onto the CPU die to drastically speed up execution.

Slide 52: Evolution of Intel Microprocessors (1990s)

The 1990s marked the introduction of the famous "Pentium" brand and the transition to 64-bit data buses.

- **The Processors:** This era featured the 486 SX (1991), the original Pentium (1993), the Pentium Pro (1995), and the Pentium II (1997).
- **Speed and Width:** Clock speeds pushed well past the 100 MHz barrier, reaching 200–300 MHz by the Pentium II. While the internal architecture remained 32-bit, the external bus width expanded to 64 bits starting with the Pentium and Pentium Pro to move data faster.
- **Transistor Density:** Feature sizes shrank below 1 micron (reaching 0.35 microns on the Pentium II), allowing Intel to pack 7.5 million transistors onto a single chip.
- **Memory Capabilities:** Addressable physical memory took another massive leap, growing from 4 GB to 64 GB with the Pentium Pro. Cache structures became more complex, with the Pentium Pro featuring dedicated 512 kB L1 and 1 MB L2 caches to keep the increasingly fast processors fed with data.

	486TM SX	Pentium	Pentium Pro	Pentium II
Introduced	1991	1993	1995	1997
Clock speeds	16 MHz - 33 MHz	60 MHz - 166 MHz,	150 MHz - 200 MHz	200 MHz - 300 MHz
Bus width	32 bits	32 bits	64 bits	64 bits
Number of transistors	1.185 million	3.1 million	5.5 million	7.5 million
Feature size (µm)	1	0.8	0.6	0.35
Addressable memory	4 GB	4 GB	64 GB	64 GB
Virtual memory	64 TB	64 TB	64 TB	64 TB
Cache	8 kB	8 kB	512 kB L1 and 1 MB L2	512 kB L2

Slide 53: Evolution of Intel Microprocessors (Recent)

This slide outlines the modern era of microprocessors, marked by the shift from Megahertz (MHz) to Gigahertz (GHz) and the introduction of multi-core designs.

- **The Processors:** This includes the Pentium III (1999), Pentium 4 (2000), Core 2 Duo (2006), and the Core i7 EE 4960X (2013).
- **Clock Speeds and Cores:** Speeds exploded from 450 MHz on the Pentium III to 4 GHz on the Core i7. More importantly, because pushing clock speeds generated too much heat, Intel shifted to adding multiple processing cores to a single chip; the Core 2 Duo featured 2 cores, and the Core i7 EE packed 6 cores.
- **Transistor Density:** By shrinking the manufacturing process down to an incredibly microscopic 22 nanometers (nm), transistor counts surged from 9.5 million to an astonishing 1.86 billion.
- **Memory:** While addressable and virtual memory caps remained steady at 64 GB and 64 TB respectively, the L2 and L3 cache sizes ballooned, with the Core i7 featuring 1.5 MB of L2 and 15 MB of L3 cache to manage data sharing between its six cores.

	Pentium III	Pentium 4	Core 2 Duo	Core i7 EE 4960X
Introduced	1999	2000	2006	2013
Clock speeds	450 - 660 MHz	1.3 - 1.8 GHz	1.06 - 1.2 GHz	4 GHz
Bus width	64 bits	64 bits	64 bits	64 bits
Number of transistors	9.5 million	42 million	167 million	1.86 billion
Feature size (nm)	250	180	65	22
Addressable memory	64 GB	64 GB	64 GB	64 GB
Virtual memory	64 TB	64 TB	64 TB	64 TB
Cache	512 kB L2	256 kB L2	2 MB L2	1.5 MB L2/15 MB L3
Number of cores	1	1	2	6

Slide 54: The Evolution of the Intel x86 Architecture

This slide compares the two most dominant processor families in the world today: Intel x86 and ARM.

- **Intel x86 (CISC):** The current x86 processors are the result of decades of design effort focused on Complex Instruction Set Computers (CISCs). CISC architecture relies on a vast, versatile library of instructions where a single command can perform multiple complex, low-level operations.
- **ARM (RISC):** As an alternative to Intel's approach, the ARM architecture relies on Reduced Instruction Set Computer (RISC) design principles. RISC uses a smaller, highly optimized set of instructions that execute extremely quickly and require less power.
- Because of its efficiency, the ARM architecture is used in a wide variety of embedded systems and is considered one of the most powerful and best-designed RISC-based systems on the market today.

Slide 55: Highlights of the Evolution of the Intel Product Line (Part 1)

This slide details the specific architectural milestones achieved by early Intel chips.

- **8080:** The world's first general-purpose microprocessor (an 8-bit machine with an 8-bit data path to memory). It famously powered the Altair, the first personal computer.
- **8086 / 8088:** A much more powerful 16-bit machine that introduced an instruction cache (or queue) that prefetched instructions before execution. This chip marked the very first appearance of the x86 architecture. The 8088 variant was chosen for IBM's first personal computer, cementing Intel's dominance.
- **80286:** An extension of the 8086 that expanded memory addressing from 1 MB to 16 MB.
- **80386:** Intel's very first 32-bit machine, and notably the first Intel processor equipped with dedicated hardware to support multitasking (running multiple programs at once).
- **80486:** This chip introduced much more sophisticated cache technology and advanced instruction pipelining, alongside a built-in math coprocessor for rendering graphics and doing complex calculations.

Slide 56: Highlights of the Evolution of the Intel Product Line (Part 2)

This slide continues the timeline into the modern Pentium and Core eras, focusing on advanced execution techniques.

- **Pentium:** Intel introduced "superscalar" techniques, meaning the processor had multiple execution pipelines allowing it to execute multiple distinct instructions simultaneously in parallel.
- **Pentium Pro/II/III:** These chips heavily optimized the superscalar design using aggressive new software tricks: register renaming, branch prediction (guessing which instruction comes next), data flow analysis, and speculative execution. They also incorporated MMX technology, specifically designed to process video, audio, and graphics data highly efficiently.
- **Core Architecture:** The Core architecture officially extended the x86 instruction set to 64 bits. It pushed multi-core processing into the mainstream, with the Core 2 Quad providing four processing cores on a single chip, and newer models offering up to 10 cores. It also added the Advanced Vector Extensions instruction set to handle massive parallel math operations.

Slide 57: Embedded Systems

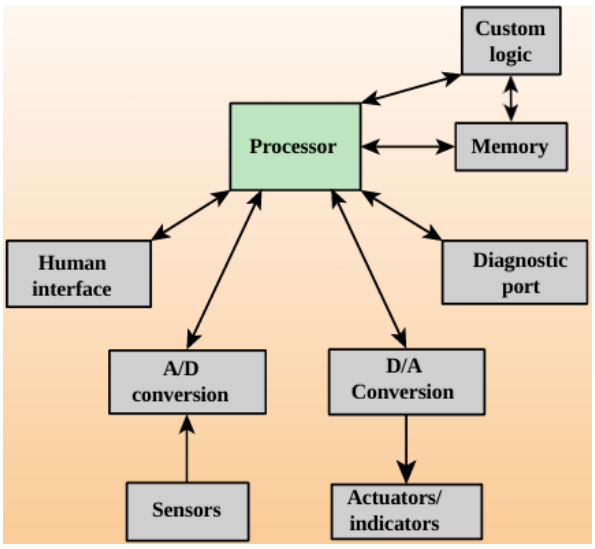
This slide transitions away from traditional desktop computers and introduces embedded systems.

- An embedded system is defined as the use of electronics and software built directly into a larger, non-computer product.
- Billions of these computing systems are produced every year. Today, almost any device that requires electric power—from washing machines to digital cameras to microwaves—has an embedded computing system inside it.
- **Real-Time Constraints:** Embedded systems are often tightly coupled to their physical environment, creating strict real-time constraints. The software must react instantly to physical inputs, dictating required speeds of motion, precise measurement intervals, and rigid time durations. Managing multiple physical activities simultaneously makes these real-time requirements highly complex.

Slide 58: Organization of an Embedded System

This diagram (Figure 1.14) visualizes the hardware architecture of a standard embedded system.

- The system is centered around a central **Processor** connected to **Memory** and optional **Custom logic** circuits.
- It interacts with the physical world through specific modules:
    - **Human interface:** Allows a user to interact with the device (e.g., the keypad on a microwave).
    - **A/D conversion (Analog to Digital):** This module takes real-world analog physical data collected by **Sensors** (like temperature, light, or pressure) and converts it into digital data the processor can read.



- **D/A Conversion (Digital to Analog):** This module takes digital commands from the processor and converts them back into physical analog signals to control physical **Actuators/indicators** (like spinning a motor or sounding an alarm).
- **Diagnostic port:** Used by engineers to test, debug, or repair the system.

Slide 59: The Internet of Things (IoT)

This slide introduces the concept of the IoT, a massive network of interconnected smart devices.

- The IoT is primarily driven by the explosion of deeply embedded devices, ranging from home appliances to tiny environmental sensors.
- The Internet has evolved through four distinct generations of deployment:
  1. **Information technology (IT):** Traditional devices like PCs, servers, and routers, primarily using wired connections.
  2. **Operational technology (OT):** Heavy machinery or medical devices with embedded IT, usually wired, managed by enterprise professionals.
  3. **Personal technology:** Consumer devices like smartphones and tablets, relying entirely on wireless connectivity.
  4. **Sensor/actuator technology (IoT):** The current fourth generation, characterized by billions of single-purpose embedded devices using wireless connectivity to function as part of a massive, globally interconnected system.

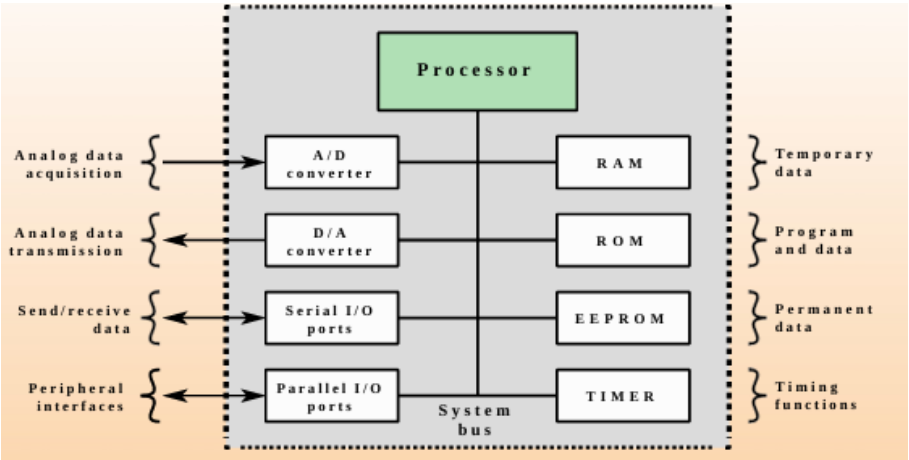
Slide 60: Embedded Operating Systems and Processors

This slide details how software and hardware are specifically tailored for embedded devices.

- **Embedded Operating Systems:** Developers essentially have two choices when creating an OS for an embedded device: they can take an existing OS (like Linux) and heavily adapt/strip it down, or they can design and implement a brand new OS intended solely for that specific embedded use.
- **Application Processors:** These are highly capable, general-purpose processors designed to execute complex, full-featured operating systems like Linux, Android, or Chrome. A smartphone processor is an application processor because it must support numerous diverse apps and wide-ranging functions.
- **Dedicated Processors:** In contrast, dedicated processors are built to perform only one (or a very small number) of specific tasks required by a host device. Because they don't need general-purpose versatility, the hardware can be heavily engineered to drastically reduce physical size and manufacturing cost.

Slide 61: Typical Microcontroller Chip Elements

This slide illustrates the anatomy of a standard microcontroller chip (Figure 1.15) and highlights the key difference between a microprocessor and a microcontroller.



- **Microprocessor vs. Microcontroller:** A microprocessor is simply a processor on a chip. A microcontroller, however,

is an entire computer built onto a single chip; it includes the microprocessor, plus on-chip memory and I/O interfaces.
- **Internal Components:** The diagram shows the central **Processor** connected via a **System bus** to:
  - **Memory:** RAM (for temporary data), ROM (for program and data), and EEPROM (for permanent data).
  - **I/O Interfaces:** A/D and D/A converters for analog data, Serial I/O ports for sending/receiving data, and Parallel I/O ports for peripheral interfaces.
  - **TIMER:** For precise timing functions.
- **Scale of Use:** Billions of microcontrollers are embedded into products every single year, ranging from simple toys to household appliances. For example, a single modern automobile can use 70 or more distinct microcontrollers to manage everything from the engine to the electric windows.

Slide 62: Deeply Embedded Systems

This slide explores a specific, highly restricted subset of embedded systems known as "deeply embedded systems".

- **Characteristics:** These systems use a microcontroller (rather than a general microprocessor) and are totally dedicated, single-purpose devices. They simply detect something in the environment, perform basic processing, and do something with the result.
- **Invisibility:** Their behavior is difficult for both programmers and users to observe. They have absolutely no screen or user interaction. Furthermore, they are not programmable once the device's logic is permanently burned into its ROM at the factory.
- **Networking:** They often feature wireless capabilities and are deployed in massive networked configurations, such as spreading hundreds of tiny sensors over a large agricultural area to monitor soil moisture.
- **Constraints:** Because they are small and often run on batteries for years, they face extreme resource constraints regarding memory size, processor speed, and power consumption.

Slide 63: ARM Architecture

This slide introduces ARM, which is arguably the most important architecture in the modern mobile and embedded world.

- **Origin:** ARM stands for Acorn RISC Machine (or Advanced RISC Machine). It is a family of microprocessors and microcontrollers designed by ARM Holdings based in Cambridge, England.
- **Design Philosophy:** The architecture evolved from RISC (Reduced Instruction Set Computer) design principles.
- **Advantages:** ARM chips are renowned for their high-speed processing combined with a very small physical die size and incredibly low power requirements (which is why they dominate the smartphone market).
- **Market Dominance:** It is likely the most widely used embedded processor architecture, and indeed, the most widely used processor architecture of *any* kind globally.

Slide 64: ARM Products

This slide visually breaks down the ARM product family into three primary categories (Cortex profiles) designed for different use cases:

- **Cortex-A / Cortex-A50:** Designed for high-end application processing (like running mobile operating systems).

- **Cortex-R:** Designed for Real-time, high-performance applications.
- **Cortex-M:** Designed specifically for Microcontroller applications, spanning from the ultra-low-power Cortex-M0 and M0+, to the more capable Cortex-M3 and M4.

Slide 65: The Cortex-A and Cortex-A50

This slide dives into the top-tier ARM processors built for heavy lifting.

- **Use Cases:** The Cortex-A and Cortex-A50 are "application processors". They are intended for mobile devices like smartphones and eBook readers, as well as consumer electronics like digital TVs and home DSL/cable modems.
- **Performance:** They run at high clock frequencies (over 1 GHz).
- **Memory Management Unit (MMU):** Crucially, they support an MMU. This is a specialized hardware module that translates virtual memory addresses into physical addresses. An MMU is an absolute requirement if a device wants to run full-featured operating systems like Linux, Android, or Windows.
- **Architecture:** Both use ARM and Thumb-2 instruction sets. The main difference is that the Cortex-A is a 32-bit machine, while the Cortex-A50 is a 64-bit machine (allowing it to process more data at once).

Slide 66: Cortex-R

This slide details the "R" series, which stands for Real-time.

- **Purpose:** The Cortex-R is designed to support real-time applications where the exact timing of events must be strictly controlled, requiring a rapid, predictable response to inputs.
- **Performance:** They run at fairly high clock speeds (200MHz to 800MHz) and feature incredibly low response latency. ARM enhanced both the instruction set and processor organization specifically to support these deeply embedded real-time tasks.
- **Memory Design:** Unlike the Cortex-A, most Cortex-R processors do not have an MMU. Because they don't run heavy OS environments with multiple simultaneous programs, they don't need elaborate virtual memory support.
- **Memory Protection Unit (MPU):** Instead of an MMU, they use an MPU. This hardware boundary ensures that one active program cannot accidentally access or overwrite the memory assigned to another program, which is critical for system stability.
- **Examples:** You'll find Cortex-R chips in zero-fail environments like automotive braking systems (where a delay of a millisecond could be fatal), mass storage controllers, and networking hardware.

Slide 67: Cortex-M Series

The "M" series is designed primarily for the microcontroller domain.

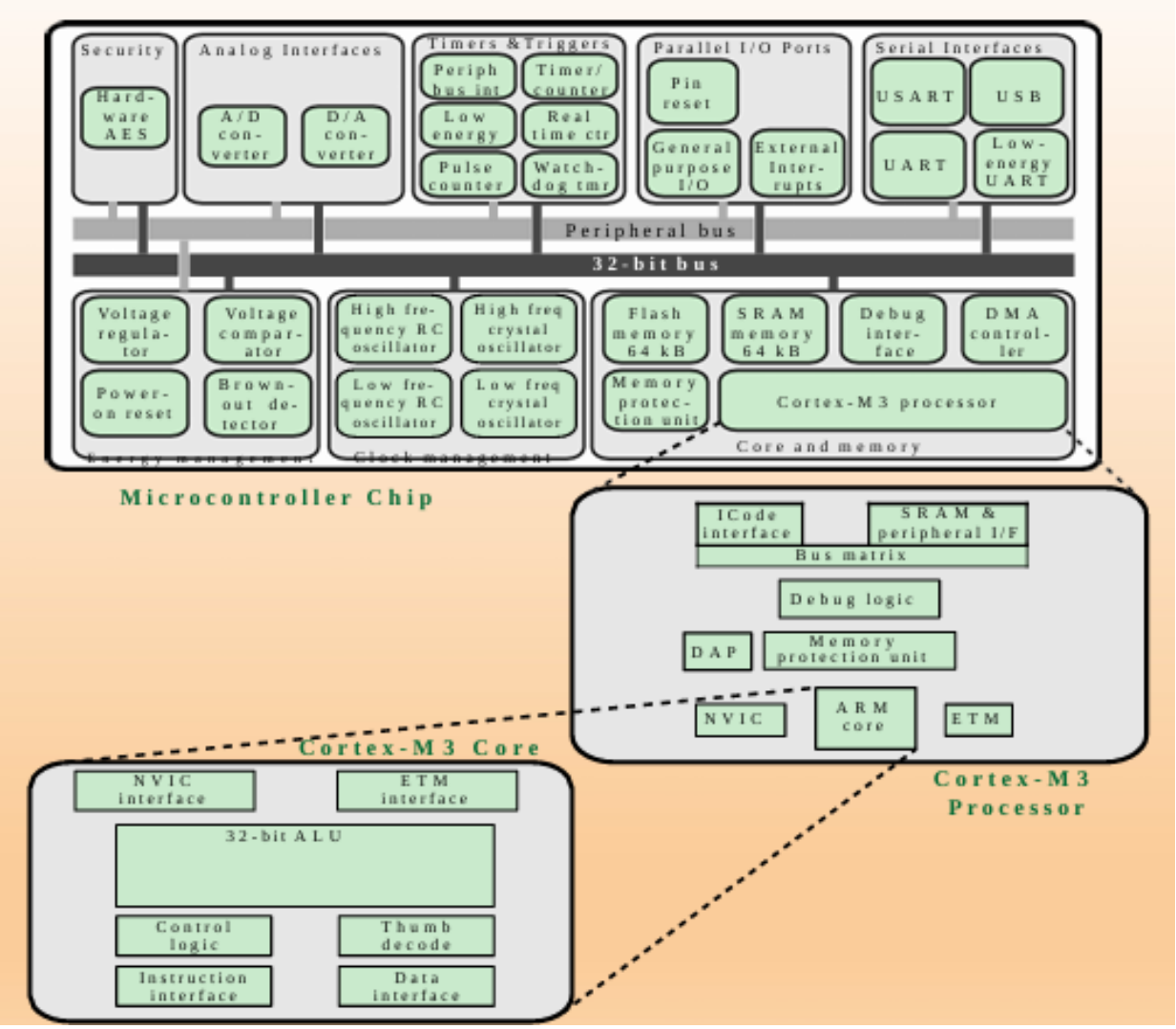
- **Purpose:** These chips balance the need for fast, highly deterministic interrupt management with the desire for extremely low gate counts and the absolute lowest possible power consumption. Like the Cortex-R, they feature an MPU but no MMU.
- **Markets:** They are heavily used in IoT devices, factory wireless sensor networks, and automotive body electronics (like windshield wiper controls).
- **The Four Versions:**
    1. **Cortex-M0:** For 8- and 16-bit tasks. It emphasizes simplicity, low cost, and ultra-low power, starting with a tiny silicon die size of just 12,000 gates.
    2. **Cortex-M0+:** An enhanced, even more energy-efficient version of the M0.

3. **Cortex-M3:** Steps up to 16- and 32-bit applications, balancing performance with energy efficiency, and adding comprehensive debug/trace features for software developers.
4. **Cortex-M4:** Includes all M3 features but adds specific instructions dedicated to digital signal processing (DSP) tasks (like processing audio or sensor data).

Slide 68: Typical Microcontroller Chip Based on Cortex-M3

This slide presents a highly detailed hardware schematic (Figure 1.16) of a complete microcontroller built around a Cortex-M3 core.

- **The Core:** The center right of the diagram shows the actual Cortex-M3 processor, featuring a 32-bit ALU, Thumb instruction decoder, and an integrated Memory Protection Unit.
- **On-Chip Components:** Surrounding the core are all the elements that make it a full computer on a chip:
  - **Memory:** 64 kB of Flash memory and 64 kB of SRAM.
  - **Security:** Dedicated hardware for AES encryption.
  - **Analog Interfaces:** A/D and D/A converters, and voltage comparators.
  - **Timers & Triggers:** Watchdog timers (which reset the system if it crashes), real-time counters, and oscillators.
  - **Serial/Parallel Interfaces:** USB, USART, UART, and General Purpose I/O pins for communicating with other hardware.



Slide 69: Cloud Computing

Transitioning from micro-level hardware to macro-level infrastructure, this slide introduces Cloud Computing.

- **NIST Definition:** The National Institute of Standards and Technology defines it as a model that enables ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources. These resources can be rapidly provisioned and released with minimal management effort or interaction with the service provider.
- **Benefits:**
  - Companies gain incredible economies of scale.
  - Subscribers don't have to hire IT staff to manage the servers; they get professional network and security management handled entirely by the cloud provider.
  - It is highly cost-effective because individuals or companies only pay for the exact storage capacity and computing services they actually use, scaling up or down instantly.

Slide 70: Cloud Networking and Cloud Storage

This slide breaks the cloud down into two essential functional concepts.

- **Cloud Networking:** This refers to the network capabilities and management functionality that absolutely must be in place to make cloud computing work.
  - For example, it requires provisioning extremely high-performance and high-reliability networking lines between the provider and the subscriber.
  - It encompasses linking an enterprise's local data center to the cloud, using specialized internet services, and deploying firewalls at critical points to enforce security policies.
- **Cloud Storage:** A specific subset of cloud computing focused entirely on data.
  - It consists of database storage and database applications that are hosted remotely on the cloud provider's servers rather than on local hard drives.
  - This allows small businesses and individuals to utilize massively scalable data storage and powerful database apps without ever having to buy, maintain, or manage the physical hard drives and storage infrastructure themselves

Slide 71: Alternative Information Technology Architectures

This slide features a detailed diagram (Figure 1.17) comparing four different IT computing models based on how responsibilities are divided between the client and the Cloud Service Provider (CSP).

- **Traditional IT architecture:** In this on-premises model, the client manages the entire technology stack. This includes everything from the physical networking, storage, and server hardware, all the way up through the virtual machines, operating system, databases, and applications. This approach is the most complex, has the highest upfront costs, and is the least scalable, but it offers the highest level of customizability.
- **Infrastructure as a Service (IaaS):** Here, the CSP takes over the foundational physical layer: networking, storage, server hardware, and virtual machines. The client is then responsible for managing the operating system, databases, run-time environment, compilers, application frameworks, and the applications themselves.
- **Platform as a Service (PaaS):** The CSP manages even more of the stack, handling everything from the networking up through the operating system, databases, run-time environments, and compilers. The client only has to worry about building and managing the application frameworks and the applications.
- **Software as a Service (SaaS):** In this fully managed model, the CSP handles absolutely everything in the stack, from the physical hardware all the way to the final applications. An example is a web-based email client. This is the least complex option, requires the lowest

upfront cost, and is incredibly scalable, but it provides the least amount of customizability for the client.

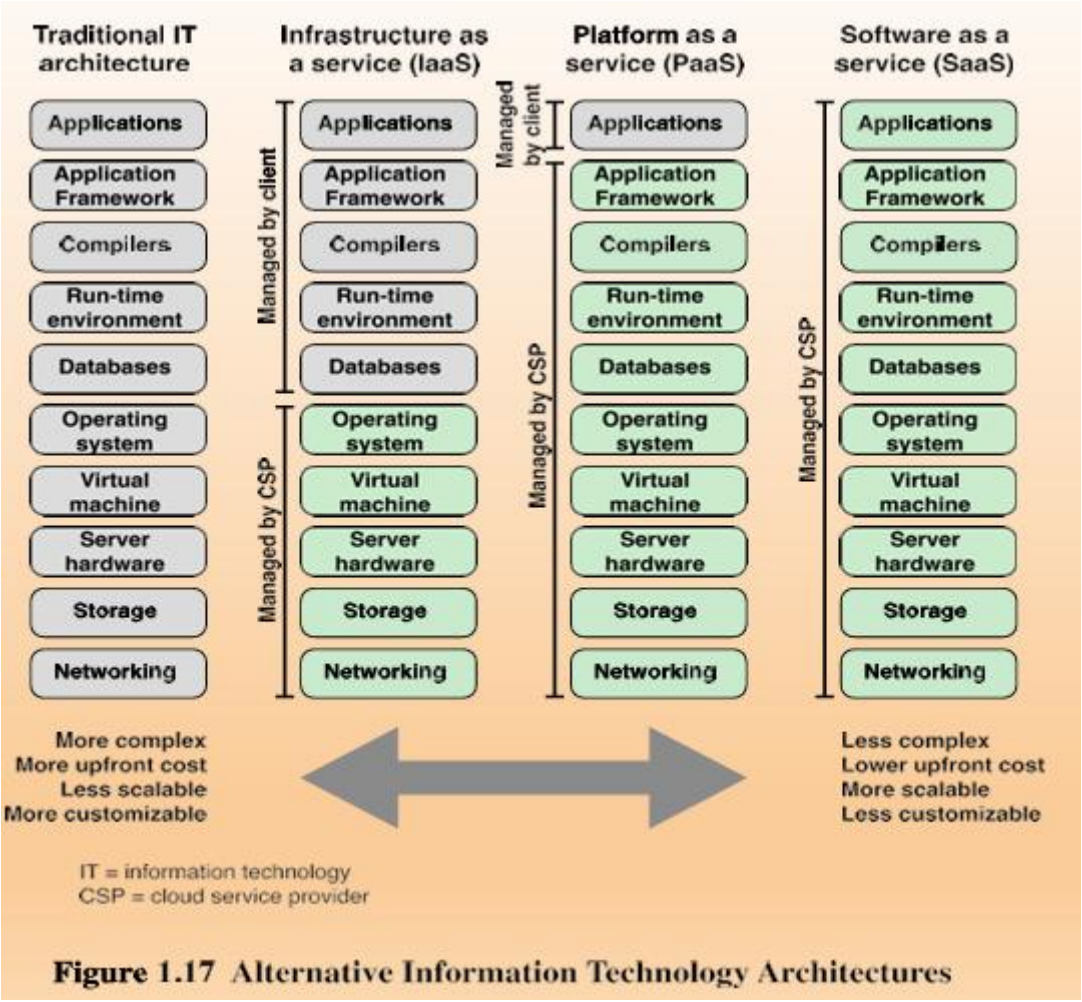


Figure 1.17 Alternative Information Technology Architectures

Slide 72: Chapter 1 Summary

This final slide serves as a comprehensive recap of everything covered in Chapter 1: "Basic Concepts and Computer Evolution". It outlines all the major themes you've learned:

- The fundamental definitions and differences between computer **Organization and architecture**, as well as their **Structure and function**.
- A **Brief history of computers**, breaking down the monumental shifts through different technological eras:
  - **The First Generation:** Vacuum tubes.
  - **The Second Generation:** Transistors.
  - **The Third Generation:** Integrated Circuits.
  - **Later generations:** The continuous shrinking of transistors (LSI, VLSI, ULSI).
- The specific **evolution of the Intel x86 architecture**, tracing it from early microprocessors to modern multi-core giants.
- The basic concepts of **Cloud computing**, including different IT architecture models.
- A deep dive into **Embedded systems**, covering:
  - **The Internet of things (IoT)**.
  - **Embedded operating systems**.
  - The difference between **Application processors versus dedicated processors**.
  - The distinction between **Microprocessors versus microcontrollers**.
  - The contrast between standard **Embedded versus deeply embedded systems**.
  - The **ARM architecture** and its specific evolution to power modern mobile and embedded devices.